

需求管理系统 PRD - 产品需求文档

1. 产品概述

1.1 产品背景

现代软件开发项目面临需求管理复杂、需求与实现组件关系不透明、跟踪效率低下等挑战。传统的静态文档和表格管理方式已无法满足日益复杂的项目需求。

1.2 产品目标

构建一个现代化的Web端需求管理系统，通过可视化界面实现：

- 高效的需求录入与管理
- 直观的需求与软件组件关联
- 可视化的项目全貌展示
- 智能化的需求检索与分析

1.3 技术架构

- 前端框架: Next.js 14+ (React 18+)
- 后端服务: Supabase (PostgreSQL + RESTful API + 实时订阅)
- 部署平台: Vercel
- UI组件库: shadcn/ui + Tailwind CSS
- 可视化: D3.js / React Flow
- 拖拽交互: dnd kit

2. 核心功能需求

2.1 功能模块概览

- 需求管理模块 - 需求的录入、编辑、导入
- 组件管理模块 - 软件组件的创建与可视化布局
- 关联管理模块 - 需求与组件的分配关联
- 搜索分析模块 - 智能搜索与关系可视化

2.2 用户角色

- 项目经理: 完整的系统访问权限，负责需求规划和组件架构
- 开发人员: 查看需求和组件，更新开发状态
- 产品经理: 创建和编辑需求，分配需求优先级

3. 详细功能规格

3.1 需求管理模块

3.1.1 手动创建需求

用户故事: 作为项目经理, 我希望能够手动创建需求条目, 以便记录和管理项目需求。

功能描述:

- 提供需求创建表单, 包含以下字段:
 - 需求标题 (必填, 最大255字符)
 - 需求描述 (富文本编辑器)
 - 优先级 (高/中/低)
 - 状态 (待分析/已分析/开发中/已完成/已废弃)
 - 创建人和创建时间 (自动记录)
 - 标签 (可多选)

验收标准:

- ☐ 表单校验正确, 必填字段不能为空
- ☐ 成功创建后显示成功提示并刷新需求列表
- ☐ 支持富文本编辑需求描述

3.1.2 批量导入需求

用户故事: 作为项目经理, 我希望能够从Excel/CSV文件批量导入需求, 以便快速迁移现有需求数据。

功能描述:

- 支持上传 .csv 和 .xlsx 格式文件
- 提供模板文件下载
- 导入前预览数据映射
- 显示导入结果统计

文件格式要求:

标题, 描述, 优先级, 状态, 标签
登录功能, 用户可以通过邮箱密码登录, 高, 待分析, "用户管理,安全"

验收标准:

- ☐ 支持拖拽上传文件
- ☐ 文件格式验证和大小限制 (最大10MB)

- ☐ 数据预览和字段映射确认
- ☐ 导入结果反馈 (成功/失败条数及详细信息)

3.2 组件管理模块

3.2.1 创建软件组件

用户故事: 作为系统架构师, 我希望能够创建代表软件模块的组件, 以便可视化系统架构。

功能描述:

- 组件基本属性:
 - 组件名称 (必填)
 - 组件类型 (前端/后端/数据库/外部服务/其他)
 - 负责人
 - 开发状态 (未开始/开发中/测试中/已完成)
 - 自定义属性 (JSON格式)

验收标准:

- ☐ 快速创建组件 (双击画布或点击添加按钮)
- ☐ 组件属性面板可编辑
- ☐ 不同类型组件有不同的视觉样式

3.2.2 可视化布局

用户故事: 作为系统架构师, 我希望能够通过拖拽方式调整组件布局, 以便清晰展示系统架构。

功能描述:

- 画布区域支持组件自由拖放
- 组件位置实时保存
- 支持缩放和平移画布
- 网格对齐辅助

技术实现:

javascript

```
// 使用 @dnd-kit/core 实现拖拽
import { DndContext, DragEndEvent } from '@dnd-kit/core';

// 组件位置保存到 Supabase
const updateComponentPosition = async (id, x, y) => {
  await supabase
    .from('components')
    .update({ position_x: x, position_y: y })
    .eq('id', id);
};
```

验收标准:

- ☐ 拖拽流畅，实时预览
- ☐ 位置变更自动保存
- ☐ 多组件框选和批量移动
- ☐ 画布缩放和平移

3.3 关联管理模块

3.3.1 需求分配

用户故事: 作为项目经理，我希望能够将需求分配给相应的软件组件，以便明确实现责任。

功能描述:

- 支持从需求列表拖拽到组件
- 支持多选模式批量分配
- 支持一个需求分配给多个组件
- 提供分配历史记录

交互方式:

1. 拖拽分配：从需求列表拖拽需求到目标组件
2. 右键菜单：右键需求选择"分配到组件"
3. 批量操作：选中多个需求批量分配

验收标准:

- ☐ 多种分配方式都可正常工作
- ☐ 分配关系立即可视化显示
- ☐ 支持取消分配操作

3.3.2 关系可视化

用户故事: 作为开发人员, 我希望能够清楚看到需求与组件的关联关系, 以便理解开发任务。

功能描述:

- 用连线显示需求-组件关联关系
- 连线样式区分不同关系类型
- 支持隐藏/显示连线
- 连线跟随组件移动

技术实现:

```
javascript

// 使用 React Flow 实现节点连线
import ReactFlow, {
  Background,
  Controls,
  MiniMap,
  ConnectionLineType
} from 'reactflow';

const connectionLineStyle = {
  strokeWidth: 2,
  stroke: '#ff6b6b',
};
```

验收标准:

- ☐ 连线清晰美观, 易于区分
- ☐ 组件移动时连线跟随
- ☐ 支持连线样式定制

3.4 搜索分析模块

3.4.1 智能搜索

用户故事: 作为用户, 我希望能够快速搜索需求, 并看到相关的组件高亮显示。

功能描述:

- 全文搜索需求标题和描述
- 支持按标签、状态、优先级筛选
- 实时搜索 (防抖处理)
- 搜索历史记录

搜索界面:

- 顶部搜索栏
- 高级筛选面板
- 搜索结果列表
- 搜索历史下拉

验收标准:

- ☐ 搜索响应时间 < 500ms
- ☐ 支持搜索语法 (例如: status:待分析)
- ☐ 搜索结果准确相关

3.4.2 关联高亮

用户故事: 作为用户，当我搜索需求时，我希望看到相关组件在画布上高亮显示，以便快速了解影响范围。

功能描述:

- 搜索结果对应的组件高亮显示
- 非相关组件变暗显示
- 高亮连线显示关联关系
- 支持多选高亮

视觉效果:

- 高亮组件：边框加粗、发光效果
- 非相关组件：透明度50%
- 连线：搜索相关的连线高亮显示

验收标准:

- ☐ 高亮效果明显易识别
- ☐ 清除搜索后恢复正常显示
- ☐ 动画过渡自然流畅

4. 数据库设计

4.1 数据表结构 (Supabase PostgreSQL)

requirements (需求表)

sql

```
CREATE TABLE requirements (  
  id BIGSERIAL PRIMARY KEY,  
  title VARCHAR(255) NOT NULL,  
  description TEXT,  
  priority VARCHAR(20) CHECK (priority IN ('高', '中', '低')),  
  status VARCHAR(20) CHECK (status IN ('待分析', '已分析', '开发中', '已完成', '已废弃')),  
  tags TEXT[], -- PostgreSQL 数组类型  
  created_by UUID REFERENCES auth.users(id),  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),  
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
);
```

components (组件表)

```
sql  
  
CREATE TABLE components (  
  id BIGSERIAL PRIMARY KEY,  
  name VARCHAR(255) NOT NULL,  
  type VARCHAR(50) CHECK (type IN ('前端', '后端', '数据库', '外部服务', '其他')),  
  position_x INTEGER DEFAULT 0,  
  position_y INTEGER DEFAULT 0,  
  width INTEGER DEFAULT 200,  
  height INTEGER DEFAULT 100,  
  owner_id UUID REFERENCES auth.users(id),  
  status VARCHAR(20) CHECK (status IN ('未开始', '开发中', '测试中', '已完成')),  
  custom_properties JSONB DEFAULT '{}',  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),  
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
);
```

requirement_component_links (关联关系表)

```
sql  
  
CREATE TABLE requirement_component_links (  
  id BIGSERIAL PRIMARY KEY,  
  requirement_id BIGINT REFERENCES requirements(id) ON DELETE CASCADE,  
  component_id BIGINT REFERENCES components(id) ON DELETE CASCADE,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),  
  UNIQUE(requirement_id, component_id)  
);
```

4.2 RLS (Row Level Security) 策略

```
sql

-- 启用 RLS
ALTER TABLE requirements ENABLE ROW LEVEL SECURITY;
ALTER TABLE components ENABLE ROW LEVEL SECURITY;
ALTER TABLE requirement_component_links ENABLE ROW LEVEL SECURITY;

-- 创建访问策略 (简化版, 实际需要根据角色细化)
CREATE POLICY "用户可以查看所有需求" ON requirements FOR SELECT USING (true);
CREATE POLICY "用户可以创建需求" ON requirements FOR INSERT WITH CHECK (auth.uid() = created_by);
```

5. API 设计 (Supabase 自动生成)

5.1 需求相关 API

```
javascript

// 获取需求列表
const { data: requirements } = await supabase
  .from('requirements')
  .select('*')
  .order('created_at', { ascending: false });

// 创建需求
const { data, error } = await supabase
  .from('requirements')
  .insert({
    title: '新需求',
    description: '需求描述',
    priority: '中',
    status: '待分析',
    tags: ['标签1', '标签2']
  });

// 搜索需求
const { data } = await supabase
  .from('requirements')
  .select('*', requirement_component_links(component_id)')
  .textSearch('title', query)
  .ilike('description', `%${query}%`);
```

5.2 实时订阅

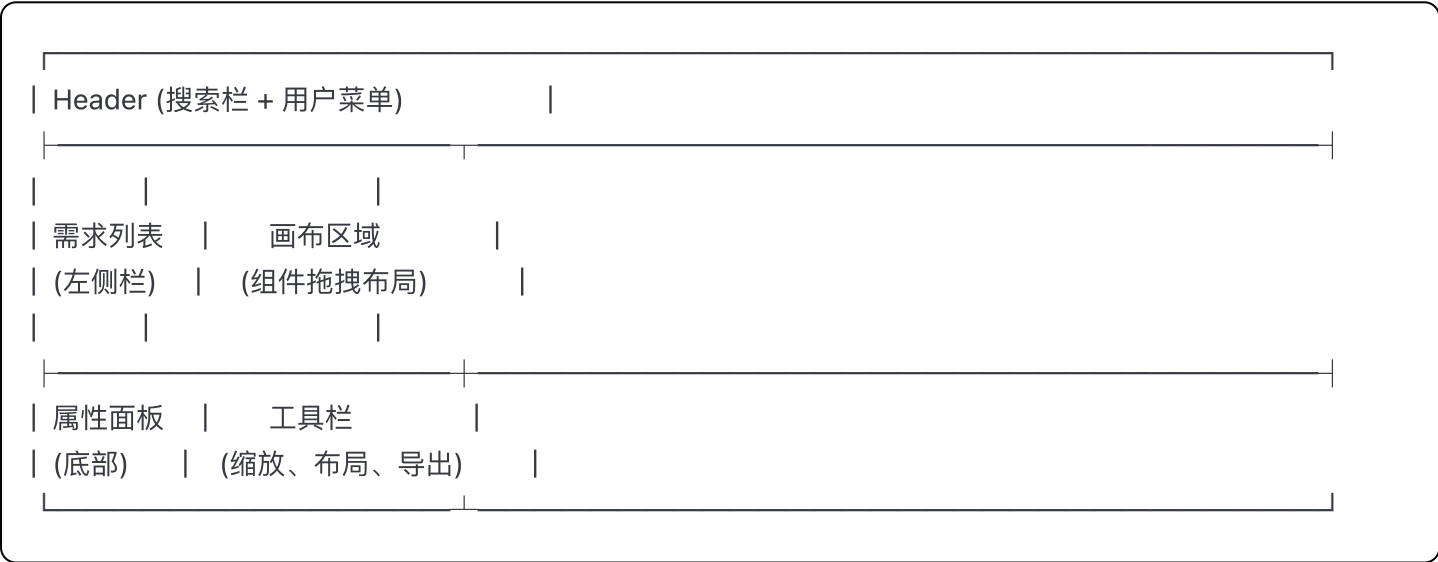
```
javascript
```



```
// 监听需求变更
const channel = supabase
  .channel('requirements')
  .on('postgres_changes',
    { event: '*', schema: 'public', table: 'requirements' },
    (payload) => {
      console.log('需求变更:', payload);
      // 更新UI状态
    }
  )
  .subscribe();
```

6. UI/UX 设计要求

6.1 整体布局



6.2 视觉设计要求

- 设计风格: 现代简约、扁平化设计
- 色彩方案:
 - 主色调: #3B82F6 (蓝色)
 - 成功: #10B981 (绿色)
 - 警告: #F59E0B (橙色)
 - 危险: #EF4444 (红色)
- 字体: Inter / 苹方-简 (中文)
- 响应式: 支持桌面端，最小宽度1200px

6.3 交互设计要求

- 操作反馈及时 (loading状态、toast提示)

- 拖拽操作流畅自然
- 快捷键支持 (Ctrl+S保存、Ctrl+Z撤销等)
- 右键上下文菜单

7. 技术实现计划

7.1 项目结构

```
needs-management/
├── app/                # Next.js 13+ App Router
│   ├── (dashboard)/   # 路由组
│   │   ├── layout.tsx
│   │   ├── page.tsx    # 主面板页面
│   │   └── settings/   # 设置页面
│   ├── api/           # API路由 (如需要)
│   ├── globals.css
│   └── layout.tsx
├── components/        # React组件
│   ├── ui/            # 基础UI组件
│   ├── requirement-list.tsx
│   ├── component-canvas.tsx
│   └── search-panel.tsx
├── lib/               # 工具库
│   ├── supabase.ts    # Supabase客户端
│   ├── types.ts       # TypeScript类型定义
│   └── utils.ts
├── hooks/             # 自定义hooks
└── public/            # 静态资源
```

7.2 关键技术栈版本

- Next.js: ^14.0.0
- React: ^18.0.0
- Supabase: ^2.38.0
- TypeScript: ^5.0.0
- Tailwind CSS: ^3.3.0
- @dnd-kit/core: ^6.0.0
- reactflow: ^11.10.0

7.3 开发环境配置

```
bash
```

项目初始化

```
npx create-next-app@latest needs-management --typescript --tailwind --app
```

安装依赖

```
npm install @supabase/supabase-js @dnd-kit/core @dnd-kit/sortable
```

```
npm install reactflow lucide-react @radix-ui/react-dialog
```

```
npm install @hookform/resolvers react-hook-form zod
```

8. 部署和运维

8.1 Vercel 部署配置

```
json
// vercel.json
{
  "buildCommand": "npm run build",
  "devCommand": "npm run dev",
  "framework": "nextjs",
  "env": {
    "NEXT_PUBLIC_SUPABASE_URL": "@supabase-url",
    "NEXT_PUBLIC_SUPABASE_ANON_KEY": "@supabase-anon-key"
  }
}
```

8.2 环境变量

```
bash
# .env.local
NEXT_PUBLIC_SUPABASE_URL=your_supabase_url
NEXT_PUBLIC_SUPABASE_ANON_KEY=your_supabase_anon_key
SUPABASE_SERVICE_ROLE_KEY=your_service_role_key
```

9. 项目里程碑

Phase 1: 基础功能 (4周)

- ☐ 项目初始化和基础架构搭建
- ☐ 用户认证系统 (Supabase Auth)
- ☐ 需求管理模块 (CRUD)
- ☐ 组件管理模块 (CRUD)
- ☐ 基础拖拽布局功能

Phase 2: 核心功能 (4周)

- ☐ 需求-组件关联功能
- ☐ 可视化连线显示
- ☐ 批量导入功能
- ☐ 基础搜索功能

Phase 3: 增强功能 (3周)

- ☐ 智能高亮显示
- ☐ 高级搜索和筛选
- ☐ 实时协作 (Supabase实时订阅)
- ☐ 导出功能

Phase 4: 优化完善 (2周)

- ☐ 性能优化
- ☐ UI/UX 优化
- ☐ 用户反馈收集和改进
- ☐ 部署和发布

10. 风险评估

10.1 技术风险

- 拖拽性能:** 大量组件时可能出现性能问题
- 实时同步:** 多用户协作时的数据同步复杂度
- 浏览器兼容性:** 复杂交互在不同浏览器的表现差异

10.2 缓解措施

- 使用虚拟滚动优化长列表性能
- 实施乐观更新减少网络延迟感知
- 渐进式功能增强，确保核心功能在所有浏览器正常工作

*此PRD文档基于原研究报告分析整理，采用Next.js + Supabase + Vercel技术栈重新设计